

2303A51204

Batch:04

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week1 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number: 1.3(Present assignment number)/24(Total number of assignments)																				
Q.No.	Question	Expected Time to complete																		
1	Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI	Week1 - Monday																		

Lab Objectives:

- ❖ To explore and evaluate the functionality of Google Gemini for AI-assisted coding within Google Colab.
- ❖ To understand and use Cursor AI for code generation, explanation, and refactoring.
- ❖ To compare outputs and usability between Gemini, GitHub Copilot, and Cursor AI.
- ❖ To perform code optimization and documentation using AI tools.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- ❖ Generate Python code using Google Gemini in Google Colab.
- ❖ Analyze the effectiveness of code explanations and suggestions by Gemini.
- ❖ Set up and use Cursor AI for AI-powered coding assistance.
- ❖ Evaluate and refactor code using Cursor AI features.
- ❖ Compare AI tool behavior and code quality across different platforms.

Task 1: Refactoring Odd/Even Logic (List Version)

- ❖ **Scenario:**
You are improving legacy code.
- ❖ **Task:**
Write a program to calculate the sum of odd and even numbers in a list, then refactor it using AI.
- ❖ **Expected Output:**
Original and improved code

✓ Original Code

```
[1]
✓ 0s  def calculate_sums_original(numbers):
      odd_sum = 0
      even_sum = 0
      for num in numbers:
          if num % 2 == 0:
              even_sum += num
          else:
              odd_sum += num
      return odd_sum, even_sum

# Example usage
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
odd_total, even_total = calculate_sums_original(my_list)
print(f"Original Code - Odd sum: {odd_total}, Even sum: {even_total}")

... Original Code - Odd sum: 25, Even sum: 30
```

Improved Code (Refactored)

```
def calculate_sums_improved(numbers):
    # Using a generator expression with sum() for a more concise and potentially efficient solution
    odd_sum = sum(num for num in numbers if num % 2 != 0)
    even_sum = sum(num for num in numbers if num % 2 == 0)
    return odd_sum, even_sum

# Example usage
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
odd_total_improved, even_total_improved = calculate_sums_improved(my_list)
print(f"Improved Code - Odd sum: {odd_total_improved}, Even sum: {even_total_improved}")

# Another improved version using list comprehension (less memory efficient than generator for very large list)
def calculate_sums_list_comp(numbers):
    odd_numbers = [num for num in numbers if num % 2 != 0]
    even_numbers = [num for num in numbers if num % 2 == 0]
    return sum(odd_numbers), sum(even_numbers)

odd_total_lc, even_total_lc = calculate_sums_list_comp(my_list)
print(f"Improved (List Comp) Code - Odd sum: {odd_total_lc}, Even sum: {even_total_lc}")
```

Improved Code - Odd sum: 25, Even sum: 30
Improved (List Comp) Code - Odd sum: 25, Even sum: 30

Task 2: Area Calculation Explanation

❖ Scenario:

You are onboarding a junior developer.

❖ Task:

Ask Gemini to explain a function that calculates the area of different shapes.

❖ Expected Output:

- Code
- Explanation

Function to be explained by Gemini

```
import math
def calculate_area(shape, dimensions):
    if shape == 'circle':
        radius = dimensions.get('radius')
        if radius is not None:
            return math.pi * radius**2
    elif shape == 'rectangle':
        length = dimensions.get('length')
        width = dimensions.get('width')
        if length is not None and width is not None:
            return length * width
    elif shape == 'triangle':
        base = dimensions.get('base')
        height = dimensions.get('height')
        if base is not None and height is not None:
            return 0.5 * base * height
    print(f"Warning: Could not calculate area for shape '{shape}' with dimensions {dimensions}")
    return None
# Example usage:
print(f"Area of circle (radius=5): {calculate_area('circle', {'radius': 5})}")
print(f"Area of rectangle (length=4, width=6): {calculate_area('rectangle', {'length': 4, 'width': 6})}")
print(f"Area of triangle (base=10, height=5): {calculate_area('triangle', {'base': 10, 'height': 5})}")
print(f"Area of unknown shape: {calculate_area('square', {'side': 5})}")
```

... Area of circle (radius=5): 78.53981633974483
Area of rectangle (length=4, width=6): 24

Output:

```
... Area of circle (radius=5): 78.53981633974483
Area of rectangle (length=4, width=6): 24
Area of triangle (base=10, height=5): 25.0
Warning: Could not calculate area for shape 'square' with dimensions {'side': 5}
Area of unknown shape: None
```

Task 3: Prompt Sensitivity Experiment

❖ **Scenario:**

You are testing how AI responds to different prompts.

❖ **Task:**

Use Cursor AI with different prompts for the same problem and observe code changes.

❖ **Expected Output:**

- **Prompt list**
- **Code variations**

```
# Alternative implementation using list comprehensions
def calculate_sums(numbers):
    """
    Calculate the sum of even numbers and the sum of odd numbers separately.

    Args:
        numbers: List of integers

    Returns:
        tuple: (sum_of_evens, sum_of_odds)
    """
    sum_even = sum(num for num in numbers if num % 2 == 0)
    sum_odd = sum(num for num in numbers if num % 2 != 0)

    return sum_even, sum_odd

# Example usage
if __name__ == "__main__":
    # Example list
    numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

    # Calculate sums
    sum_even, sum_odd = calculate_sums(numbers)

    # Display results
    print(f"Numbers: {numbers}")
    print(f"Sum of even numbers: {sum_even}")
```

Output:

No linter errors found in sum_even_odd_alternative.py

Created two versions:

1. `sum_even_odd.py` — Uses a for loop (already exists)
2. `sum_even_odd_alternative.py` — Uses list comprehensions

Both calculate the sum of even and odd numbers separately. The alternative version is more concise and Pythonic.

Example output:

```
Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Sum of even numbers: 30
Sum of odd numbers: 25
```

Task 4: Tool Comparison Reflection

❖ **Scenario:**

You must recommend an AI coding tool.

❖ **Task:**

Based on your work in this topic, compare Gemini, Copilot, and Cursor AI for usability and code quality.

❖ **Expected Output:**

Short written reflection

Comparison Reflection

Tool	Strengths	Limitations
Google Gemini	Excellent explanations, beginner-friendly, good for learning	Less real-time IDE integration
GitHub Copilot	Fast code completion, ideal for professionals	Minimal explanation
Cursor AI	Best for refactoring, prompt experiments, debugging	Requires setup

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.